Lecture 2-10 — DOM Traversal & Manipulation

Introduction

This lesson introduces DOM traversal: how to move around HTML elements using JavaScript, read and change classes, and wire up simple event listeners. We'll use a short example and explain every step for absolute beginners.

Code Example (HTML + JS)

Below is the exact code used

```
<!-- Begin example: list.html (shortened for readability) -->
<ul id="main">
    <li>
        <b>Electricals</b>
        <ul>
            <li>TV</li>
            <li>Fridge</li>
            <li>Washing Machine</li>
        </ul>
    </li>
    <li>
        <b>Electronics</b>
        <ul>
            <li>Mobile</li>
            <li>
                <i>Laptop</i>
                <ol>
                    <li>Lenovo</li>
                    <li class="selected">Dell</li>
                    <li>Samsung</li>
                </ol>
            </li>
            <li>Tablet</li>
        </ul>
    </li>
    <li>
        <b>Furniture</b>
```


Powered by SPRK Technologies

```
        <ul>
            <li>File Cabinet</li>
            <li>Table</li>
            <li>Chair</li>
        </ul>
    </li>
</ul>

<!-- Buttons to traverse the DOM -->
<button id="btnParent">Parent</button>
<button id="btnPrevSibling">Previous Sibling</button>
<button id="btnNextSibling">Next Sibling</button>
<button id="btnFirstChild">First Child</button>
<button id="btnLastChild">Last Child</button>
<button id="btnChildren">Print Children</button>

<script>
    // DOM traversal example
    var selected = document.querySelector(".selected");
    document.querySelector("#btnParent").addEventListener("click", function () {
        var parentEle = selected.parentElement;
        if (!parentEle) return;
        changeSelected(selected, parentEle);
    });
    // ...other event listeners and changeSelected function here...
</script>
<!-- End example -->
```

Output:

- **Electricals**
  - TV
  - Fridge
  - Washing Machine
- **Electronics**
  - Mobile
  - *Laptop*
    1. Lenovo
    2. Dell
    3. Samsung
  - Tablet

Powered by SPRK Technologies

- **Furniture**
  - File Cabinet
  - Table
  - Chair

Parent | Previous Sibling
Next Sibling | First Child
Last Child | Print Children

Elements and DOM

What is the DOM?

The Document Object Model (DOM) is a tree-like representation of the HTML page. Each HTML tag becomes a node (an element) you can access and change using JavaScript.

Selecting elements

- **document.querySelector(selector)**: returns the first element matching the CSS selector. Example: `document.querySelector('.selected')`.
- **document.getElementById(id)**: gets an element by its id (e.g. `document.getElementById('main')`).

Traversing the DOM

Once you have an element, you can move around the tree using properties:

- **parentElement**: the parent element of the current element. If none, it's null.
- **children**: an *HTMLCollection* of element children (not text nodes). You can index into it like an array: `el.children[0]`.
- **firstElementChild / lastElementChild**: the first/last child element directly under this element.
- **previousElementSibling / nextElementSibling**: the element siblings before or after the current element.

Powered by SPRK Technologies

Why use the "Element" variants?

There are similar properties without "Element" (like `previousSibling`) that can return text nodes (like whitespace). The `...Element...` versions skip non-element nodes, which is usually easier to work with.

Changing classes

Use `el.classList` to add/remove/toggle CSS classes:

- `el.classList.add('selected')`
- `el.classList.remove('selected')`
- `el.classList.toggle('selected')`

Event listeners

We attach behavior to buttons using `addEventListener`. When a button is clicked we compute the target element to move the selection to and call `changeSelected(old, new)` to update classes.

Safety checks

Because some elements may not exist in the direction you want to move, we check for `null` or absence before acting. Example: `if (!prevSiblingEle) return;`

Example walkthrough (code behavior)

1. Start: the code finds the element with `class="selected"` and stores it in the `selected` variable.
2. Click a navigation button (e.g. "Parent"): JS computes the new element, verifies it exists, and calls `changeSelected(selected, parentEle)`.
3. `changeSelected` removes the `selected` class from the old element, adds it to the new element, and updates the `selected` variable so next clicks continue from the new location.
4. The `Print Children` button logs the `children` collection to the console (or output area in this lesson) so you can inspect child elements by index.

Common beginner pitfalls

- Trying to access properties of null: always check an element exists before using it.

- Confusing node collections: `children` is a live HTMLCollection (updates if DOM changes), while methods like `querySelectorAll` return a static NodeList.
- Using string HTML versus live elements: showing code as text (in a `<pre>`) won't run — you need real DOM nodes to interact with.

Try this

In the output area above, click the buttons and watch which list item gets the gray/red styling. Open your browser console to see `children` output or use the printed output area.

Extra — Nodes and Node Types

Every piece of the DOM is a **Node**. Elements (like <ul>, <li>) are one type of node, but text inside elements and comments are also nodes. Working with elements is easier for most tasks; below we explain why and list the common node types.

Why we usually use Element APIs (not raw nodes)

- Element-focused properties (for example `parentElement`, `children`, `firstElementChild`) skip non-element nodes like whitespace or text nodes. This reduces surprises for beginners.
- Generic node properties such as `parentNode` and `childNodes` include text nodes and comments, which you rarely want when manipulating structure or classes.
- Element APIs give nicer collections and helper methods (like `classList`), so they are simpler and less error-prone.

Common node types

- **Element node** (Element): represents HTML tags. *nodeType* = 1.
- **Text node**: the text inside elements. Whitespace and line breaks create text nodes. *nodeType* = 3.
- **Comment node**: HTML comments like `<!-- comment -->`. *nodeType* = 8.
- **Document node**: the whole document (document). *nodeType* = 9.
- **DocumentFragment**: an efficient lightweight container for building parts of DOM off-screen. *nodeType* = 11.
- **Attribute nodes**: historically attributes were nodes (attr), but you usually access attributes via `el.getAttribute()` or `el.setAttribute()` instead of treating them as nodes.

Powered by SPRK Technologies

Numeric nodeType constants (quick reference)

These are built into the DOM:

- `Node.ELEMENT_NODE === 1`
- `Node.TEXT_NODE === 3`
- `Node.COMMENT_NODE === 8`
- `Node.DOCUMENT_NODE === 9`
- `Node.DOCUMENT_FRAGMENT_NODE === 11`

Small examples

Compare `childNodes` vs `children` on the live list above. Open your browser console or click "Print Children" to see the difference. Example code:

```
// show childNodes and children for the main list
console.log('childNodes:', main.childNodes); // may include text nodes (whitespace)
console.log('children:', main.children);     // only element children

// check type of a node
var first = main.childNodes[0];
console.log('nodeType:', first.nodeType, 'isElement?', first.nodeType === Node.ELEMENT_NODE);
```

Beginners: when you want to find, style, or add/remove elements, prefer the *Element* APIs (like `children`, `firstElementChild`, `parentElement`, and `querySelector`). Use node-level APIs only when you need to handle text nodes, comments, or do very low-level DOM work.

Powered by SPRK Technologies